La Citadelle

Accueillir les visiteurs

Les systèmes de sécurité sont rétablis, et la Citadelle rouvre ses portes.

Une délégation de visiteurs se présente devant Turing. Avant de les laisser entrer, le Warden veut s'assurer que chacun est digne de confiance… et capable de suivre ses raisonnements.

> « Je ne discute qu'avec ceux qui peuvent me suivre. Un diplôme ne prouve pas tout, mais c'est un début. »

Chaque visiteur présente une **université** et un **titre** (`"Bachelor"` ou `"Master"`).

Les règles du Warden :

- l'université doit figurer dans la liste des universités reconnues ;
- tout visiteur doit posséder **au moins un Bachelor** ;
- certaines universités sont si peu exigeantes que le Warden demande **un Master**.

Vous allez utiliser :

- `if` avec des conditions à plusieurs opérandes (`&&`, `||`, `!`) ;
- `else if` ;
- `switch` ;
- `for`.

Instructions

Complétez les fonctions du fichier `visiteurs.js`.

Le fichier contient déjà deux listes :

```
export const UNIVERSITIES = [
  'Académie Babbage',
  'Institut Lovelace',
  'Université Hopper',
  'École Dijkstra',
  'Collège Von Neumann',
]

export const MASTER_REQUIRED = ['Institut Lovelace', 'École Dijkstra']
```

Une liste se parcourt avec une boucle `for` classique :

```
for (let i = 0; i < UNIVERSITIES.length; i++) {
  // UNIVERSITIES[i] est l'université numéro i
}
```

1. Reconnaître une université

Implémentez :

```
isKnownUniversity(university)
```

La fonction retourne `true` si `university` figure dans `UNIVERSITIES`, sinon `false`.

Utilisez une boucle `for`.

```
isKnownUniversity('Académie Babbage')
// => true

isKnownUniversity('Université Fantôme')
// => false
```

2. Convertir un titre en niveau

Implémentez :

```
titleLevel(title)
```

Règles :

- `"Bachelor"` → `1`
- `"Master"` → `2`
- tout autre titre → `0`

Utilisez `switch`.

```
titleLevel('Master')
// => 2

titleLevel('Doctorat')
// => 0
```

3. Identifier les universités peu exigeantes

Implémentez :

```
requiresMaster(university)
```

La fonction retourne `true` si `university` figure dans `MASTER_REQUIRED`.

```
requiresMaster('Institut Lovelace')
// => true

requiresMaster('Académie Babbage')
// => false
```

4. Décider si un visiteur est digne de confiance

Implémentez :

```
isTrusted(university, title)
```

Un visiteur est digne de confiance si **toutes** ces conditions sont réunies :

- son université est reconnue ;
- son titre est au moins un Bachelor ;
- si son université exige un Master, son titre est un Master.

Écrivez **une seule condition** `if` qui combine ces règles avec `&&`, `||` et `!`.

```
isTrusted('Académie Babbage', 'Bachelor')
// => true

isTrusted('Institut Lovelace', 'Bachelor')
// => false

isTrusted('Institut Lovelace', 'Master')
// => true
```

5. Expliquer la décision

Implémentez :

```
welcomeMessage(university, title)
```

Le Warden explique toujours ses refus. Testez les règles **dans cet ordre** :

1. université inconnue → `"Université inconnue"`
2. titre inconnu → `"Titre inconnu"`
3. Master exigé mais absent → `"Master requis"`
4. sinon → `"Bienvenue dans la Citadelle"`

Utilisez `if` / `else if` / `else`.

```
welcomeMessage('École Dijkstra', 'Bachelor')
// => "Master requis"

welcomeMessage('Université Hopper', 'Bachelor')
// => "Bienvenue dans la Citadelle"
```

6. Contrôler toute la délégation

Implémentez :

```
allTrusted(universities, titles)
```

Les deux listes ont la même longueur : le visiteur numéro `i` vient de `universities[i]` et possède le titre `titles[i]`.

La fonction retourne `true` si **tous** les visiteurs sont dignes de confiance, sinon `false`.

Utilisez une boucle `for` et réutilisez `isTrusted`.

```
allTrusted(['Académie Babbage', 'Institut Lovelace'], ['Bachelor', 'Master'])
// => true

allTrusted(['Académie Babbage', 'Institut Lovelace'], ['Bachelor', 'Bachelor'])
// => false
```

À rechercher si nécessaire

- `JavaScript for loop array`
- `JavaScript logical operators`
- `JavaScript switch`
- `JavaScript else if`

▶ Aides